

AWS IAM — Users, Groups, Roles, Policies, Permission Boundaries, and MFA

Complete Guide with Detailed Examples and Top 5 Interview Questions

AWS Security Deep Dive

1970-01-01

Overview

AWS Identity and Access Management (IAM) is a web service that enables you to manage access to AWS resources securely. IAM allows you to create and manage AWS users and groups, and use permissions to allow and deny their access to AWS resources[1].

Key Characteristics

- **Centralized Access Control:** Manage user permissions from a single console
- **Fine-Grained Permissions:** Specify exactly which resources users can access
- **No Additional Charge:** IAM is a free service included with AWS account
- **Integration with AWS Services:** Works with all AWS services
- **Compliance Ready:** HIPAA, PCI-DSS, SOC 2 compliant
- **Audit Trail:** Full integration with CloudTrail for compliance

Benefits

- Secure control over AWS resources
- Principle of least privilege enforcement
- Multi-factor authentication support
- Federated identity (Active Directory, SAML)
- Cross-account access management
- Temporary credentials via STS

Core Concepts

1. IAM User

An IAM user is an identity you create in AWS for a person or application.

User Characteristics

Attribute	Description
Username	Unique identifier within AWS account
Console Password	For AWS Management Console access
Access Keys	For programmatic access (AWS CLI, API)
MFA Device	Multi-factor authentication device
Permissions	Directly attached or inherited from groups

Creating an IAM User

```
# Create IAM user
```

```
aws iam create-user \  
  --user-name john.doe \  
  --tags Key=Department,Value=Engineering Key=Project,Value=WebApp
```

```
# Response:
```

```
# {  
#   "User": {  
#     "UserName": "john.doe",  
#     "UserId": "AIDAI2ABCDEFGHIJK",  
#     "Arn": "arn:aws:iam::123456789012:user/john.doe",  
#     "Path": "/",  
#     "CreateDate": "2025-12-02T10:30:00Z"  
#   }  
# }
```

```
# Attach inline policy directly to user
```

```
aws iam put-user-policy \  
  --user-name john.doe \  
  --policy-name S3ReadPolicy \  
  --policy-document '{  
    "Version": "2012-10-17",  
    "Statement": [{  
      "Effect": "Allow",  
      "Action": "s3:Get*",  
      "Resource": "arn:aws:s3:::my-bucket/*"  
    }]  
  }'
```

```
# List users
```

```
aws iam list-users \
  --query 'Users[*].[UserName,Arn,CreateDate]' \
  --output table
```

2. IAM Group

A group is a collection of IAM users with shared permissions.

Group Benefits

Create a group

```
aws iam create-group \
  --group-name Developers \
  --path "/engineering/"
```

Attach managed policy to group

```
aws iam attach-group-policy \
  --group-name Developers \
  --policy-arn arn:aws:iam::aws:policy/AmazonEC2FullAccess
```

Add user to group

```
aws iam add-user-to-group \
  --group-name Developers \
  --user-name john.doe
```

Verify user in group

```
aws iam get-group \
  --group-name Developers \
  --query 'Users[*].UserName' \
  --output text
```

Output: john.doe jane.smith

3. IAM Role

An IAM role is an identity with permission policies but without long-term credentials.

Key Differences: User vs Role

Feature	User	Role
Credentials	Long-term (password, access keys)	Temporary (STS tokens)
Who Uses	People or applications	AWS services or users
Use Case	Direct AWS resource access	Cross-account access, services
Assumed By	User login	AssumeRole API call

Creating an IAM Role

Create trust policy document

```
cat > trust-policy.json <<'EOF'
{
```

```

"Version": "2012-10-17",
"Statement": [
  {
    "Effect": "Allow",
    "Principal": {
      "Service": "ec2.amazonaws.com"
    },
    "Action": "sts:AssumeRole"
  }
]
}
EOF

# Create role
aws iam create-role \
  --role-name EC2-Application-Role \
  --assume-role-policy-document file://trust-policy.json \
  --description "Role for EC2 instances running web application"

# Attach policy to role
aws iam attach-role-policy \
  --role-name EC2-Application-Role \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess

# Create instance profile (for EC2)
aws iam create-instance-profile \
  --instance-profile-name EC2-App-Profile

# Add role to instance profile
aws iam add-role-to-instance-profile \
  --instance-profile-name EC2-App-Profile \
  --role-name EC2-Application-Role

# Launch EC2 with role
aws ec2 run-instances \
  --image-id ami-0c55b159cbfafa1f0 \
  --instance-type t3.micro \
  --iam-instance-profile Name=EC2-App-Profile

```

IAM Policies

Policy Structure

IAM policies are JSON documents that define permissions.

Policy Components

```

{
  "Version": "2012-10-17",

```

```

"Statement": [
  {
    "Sid": "AllowS3ReadWrite",
    "Effect": "Allow",
    "Action": [
      "s3:GetObject",
      "s3:PutObject"
    ],
    "Resource": "arn:aws:s3:::my-bucket/*",
    "Condition": {
      "StringEquals": {
        "aws:SourceIp": "203.0.113.0/24"
      }
    }
  },
  {
    "Sid": "DenyDeleteBucket",
    "Effect": "Deny",
    "Action": "s3:DeleteBucket",
    "Resource": "*"
  }
]
}

```

Component	Description
Version	Policy language version (always use 2012-10-17)
Statement	Array of individual policy statements
Sid	Optional identifier for the statement
Effect	Allow or Deny
Principal	Who the policy applies to (in resource policies)
Action	AWS API actions (e.g., s3:GetObject)
Resource	AWS resources the statement applies to
Condition	Optional conditions (IP, time, MFA)

Policy Types

1. Managed Policies

AWS Managed Policy (predefined)

```

aws iam attach-user-policy \
  --user-name john.doe \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess

```

Create Custom Managed Policy

```
aws iam create-policy \  
  --policy-name CustomS3Policy \  
  --policy-document '{  
    "Version": "2012-10-17",  
    "Statement": [{  
      "Effect": "Allow",  
      "Action": ["s3:GetObject", "s3:ListBucket"],  
      "Resource": [  
        "arn:aws:s3:::my-bucket",  
        "arn:aws:s3:::my-bucket/*"  
      ]  
    }]  
  }'
```

Attach custom managed policy

```
aws iam attach-user-policy \  
  --user-name john.doe \  
  --policy-arn arn:aws:iam::123456789012:policy/CustomS3Policy
```

2. Inline Policies

Put inline policy directly on user

```
aws iam put-user-policy \  
  --user-name john.doe \  
  --policy-name DynamoDBAccess \  
  --policy-document '{  
    "Version": "2012-10-17",  
    "Statement": [{  
      "Effect": "Allow",  
      "Action": "dynamodb:*",  
      "Resource":  
"arn:aws:dynamodb:us-east-1:123456789012:table/users"  
    }]  
  }'
```

List inline policies

```
aws iam list-user-policies --user-name john.doe
```

Delete inline policy

```
aws iam delete-user-policy \  
  --user-name john.doe \  
  --policy-name DynamoDBAccess
```

Advanced Policy Examples

Least Privilege S3 Access

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {
```

```

    "Sid": "ListBuckets",
    "Effect": "Allow",
    "Action": "s3:ListBucket",
    "Resource": "arn:aws:s3:::my-bucket"
  },
  {
    "Sid": "ReadObjects",
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::my-bucket/documents/*"
  },
  {
    "Sid": "DenyEncryptionBypass",
    "Effect": "Deny",
    "Action": "s3:PutObject",
    "Resource": "arn:aws:s3:::my-bucket/*",
    "Condition": {
      "StringNotEquals": {
        "s3:x-amz-server-side-encryption": "aws:kms"
      }
    }
  }
]
}

```

Permission Boundaries

Permission boundaries set the maximum permissions an identity can receive[1].

When to Use Permission Boundaries

- **Delegate permissions management:** Allow developers to create roles with limited permissions
- **Risk mitigation:** Prevent privilege escalation
- **Compliance:** Enforce organizational security standards

Permission Boundary Example

```

# 1. Create permission boundary policy
cat > boundary-policy.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3Access",
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": "arn:aws:s3:::dev-*"
    }
  ],

```

```

    {
      "Sid": "DynamoDBAccess",
      "Effect": "Allow",
      "Action": "dynamodb:*",
      "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/dev-
*"
    }
  ]
}
EOF

```

2. Create managed policy for boundary

```

aws iam create-policy \
  --policy-name DeveloperBoundary \
  --policy-document file://boundary-policy.json

```

3. Create IAM user and attach boundary

```

aws iam create-user --user-name intern-user

```

```

aws iam put-user-permissions-boundary \
  --user-name intern-user \
  --permissions-boundary \
  arn:aws:iam::123456789012:policy/DeveloperBoundary

```

4. Attach identity-based policy (must be within boundary)

```

aws iam put-user-policy \
  --user-name intern-user \
  --policy-name InternPolicy \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": "arn:aws:s3:::dev-bucket/*"
    }]
  }'

```

Verify boundary is set

```

aws iam get-user-permissions-boundary \
  --user-name intern-user

```

Permission Boundary Mechanism

Access Granted IF:

1. Identity-based policy allows action AND
2. Permission boundary allows action AND
3. Resource policy (if exists) allows action

Access Denied IF:

- Any of the above denies the action

Multi-Factor Authentication (MFA)

MFA Types

Type	Examples	Cost
Virtual MFA	Google Authenticator, Microsoft Authenticator, Authy	Free
Hardware MFA	YubiKey, Gemalto SafeNet	\$25-\$60
SMS-Based	Text message codes	Free (deprecated)

Enabling MFA for IAM User

Step 1: Get MFA devices for a user

```
aws iam list-mfa-devices --user-name john.doe
```

Step 2: Enable virtual MFA device

```
aws iam enable-mfa-device \  
  --user-name john.doe \  
  --serial-number arn:aws:iam::123456789012:mfa/john.doe-mfa \  
  --authentication-code1 123456 \  
  --authentication-code2 654321
```

Verify MFA is enabled

```
aws iam list-mfa-devices --user-name john.doe
```

Output shows MFA device ARN

Step 3: Deactivate MFA (if needed)

```
aws iam deactivate-mfa-device \  
  --user-name john.doe \  
  --serial-number arn:aws:iam::123456789012:mfa/john.doe-mfa
```

Enforcing MFA Requirement

Policy requiring MFA for sensitive operations

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "AllowWithMFA",  
      "Effect": "Allow",  
      "Action": [  
        "iam:CreateUser",  
        "iam>DeleteUser"  
      ],  
      "Resource": "*",  
      "Condition": {  
        "Bool": {  
          "aws:MultiFactorAuthPresent": "true"  
        }  
      }  
    }  
  ]  
}
```

```

    }
  },
  {
    "Sid": "DenyWithoutMFA",
    "Effect": "Deny",
    "Action": [
      "iam:DeleteUser",
      "iam:AttachUserPolicy"
    ],
    "Resource": "*",
    "Condition": {
      "BoolIfExists": {
        "aws:MultiFactorAuthPresent": "false"
      }
    }
  }
]
}

```

Practical Implementation Scenarios

Scenario 1: Multi-Team Organization

Requirements

- Engineering team: Full EC2 and S3 access
- Finance team: Read-only billing access
- DevOps team: Full AWS access with MFA requirement

Implementation

```
#!/bin/bash
```

1. Create groups

```
aws iam create-group --group-name Engineering
aws iam create-group --group-name Finance
aws iam create-group --group-name DevOps
```

2. Create users

```
aws iam create-user --user-name alice.johnson
aws iam create-user --user-name bob.smith
aws iam create-user --user-name charlie.brown
```

3. Add users to groups

```
aws iam add-user-to-group --group-name Engineering --user-name
alice.johnson
aws iam add-user-to-group --group-name Finance --user-name bob.smith
aws iam add-user-to-group --group-name DevOps --user-name
```

charlie.brown

```
# 4. Attach policies to groups
# Engineering: EC2 and S3 full access
aws iam attach-group-policy \
  --group-name Engineering \
  --policy-arn arn:aws:iam::aws:policy/AmazonEC2FullAccess
aws iam attach-group-policy \
  --group-name Engineering \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3FullAccess

# Finance: Billing read-only
aws iam attach-group-policy \
  --group-name Finance \
  --policy-arn arn:aws:iam::aws:policy/ViewOnlyAccess

# DevOps: Custom policy with MFA requirement
aws iam put-group-policy \
  --group-name DevOps \
  --policy-name DevOpsPolicy \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": "*",
        "Resource": "*",
        "Condition": {
          "Bool": {
            "aws:MultiFactorAuthPresent": "true"
          }
        }
      }
    ]
  }'
```

Top 5 Interview Questions

Question 1: Users vs Groups vs Roles (Fundamental)

Question

“Explain the difference between IAM Users, Groups, and Roles. When would you use each one?”

Expected Answer

IAM User:

- Represents an individual person or application

- Has long-term credentials (password, access keys)
- Direct access to AWS resources
- Used for: Developers, database administrators, CI/CD systems

IAM Group:

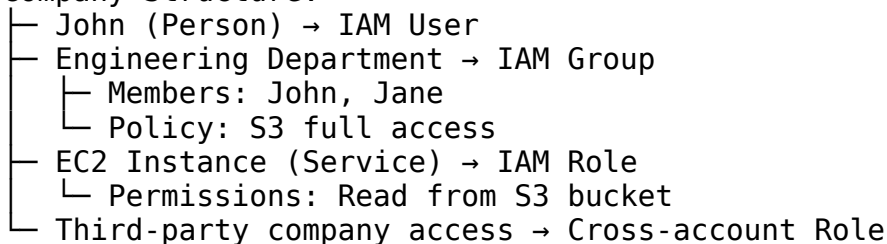
- Collection of IAM users
- Simplifies permission management
- Inherit permissions from attached policies
- Used for: Department-level access control

IAM Role:

- Identity with temporary credentials
- Assumed using AssumeRole API call
- Trust relationship defines who can assume
- Used for: Service-to-service access, cross-account access, EC2 instances

Real-World Example:

Company Structure:



Question 2: Permission Boundaries (Intermediate)

Question

“Your company needs to allow new developers to create IAM roles for their applications, but you want to prevent them from creating highly privileged roles. How would you implement this?”

Expected Answer

Solution: Use Permission Boundaries

Permission boundaries set the maximum permissions that can be granted. They act as a safety net.

Implementation Steps:

1. Create Boundary Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::app-bucket/*"
    },
    {
      "Effect": "Allow",
      "Action": "dynamodb:*",
      "Resource": "arn:aws:dynamodb:us-east-1:*:table/app-*"
    }
  ]
}
```

2. Attach Boundary to Developer

```
aws iam put-user-permissions-boundary \
  --user-name developer-user \
  --permissions-boundary \
    arn:aws:iam::123456789012:policy/DevBoundary
```

3. Allow Role Creation with Boundary

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "iam:CreateRole",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "iam:PermissionsBoundary":
            "arn:aws:iam::123456789012:policy/DevBoundary"
        }
      }
    }
  ]
}
```

Result: Developer can create roles, but any role will be limited to S3 and DynamoDB resources only.

Question 3: MFA for Root Account (Security Best Practice)

Question

“Why is it critical to enable MFA on the root account? What would happen if MFA is not enabled?”

Expected Answer

Why MFA is Critical for Root Account:

The root account has unrestricted access to all AWS resources, including:

- Delete the entire AWS account
- Close AWS account
- Delete all resources (EC2, RDS, S3)
- Change billing information
- Enable/disable MFA on other users
- Delete IAM users and policies

Risk if MFA Not Enabled:

Attacker scenario:

1. Obtains root account email and password (via phishing)
2. Logs in without MFA
3. Deletes all AWS resources
4. Changes billing contact
5. Incurs massive charges
6. Company data is lost

MFA Protection:

With MFA enabled:

1. Attacker gets password
2. Login requires MFA device code (attacker doesn't have)
3. Login fails
4. AWS sends alert to registered email
5. Account is protected

AWS Best Practice:

1. Enable MFA on root account immediately
2. Never share root credentials
3. Use root only for billing and account setup
4. Create IAM users for daily work
5. Use AWS Organizations for multi-account setup

Question 4: Cross-Account Access (Advanced)

Question

“Design a solution where developers in Account A need read access to S3 buckets in Account B. What IAM components are involved?”

Expected Answer

Architecture:



Implementation:

Account B (Production):

Step 1: Create Role with S3 permissions

```
# Trust policy - allowing Account A to assume
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::ACCOUNT-A-ID:root"
    },
    "Action": "sts:AssumeRole"
  }]
}
```

```
# Create role
aws iam create-role \
  --role-name CrossAccountS3ReadRole \
  --assume-role-policy-document file://trust.json
```

Step 2: Attach S3 read policy

```
aws iam attach-role-policy \
  --role-name CrossAccountS3ReadRole \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

Account A (Developers):

Step 3: Allow developers to assume the role

```
{
  "Version": "2012-10-17",
  "Statement": [{
```

```

    "Effect": "Allow",
    "Action": "sts:AssumeRole",
    "Resource":
        "arn:aws:iam::ACCOUNT-B-ID:role/CrossAccountS3ReadRole"
  }]
}

```

Developer Assumes Role:

```

# Assume role in Account B
aws sts assume-role \
  --role-arn arn:aws:iam::ACCOUNT-B-ID:role/CrossAccountS3ReadRole \
  --role-session-name my-session

```

```

# Response includes temporary credentials
# Developer can now use S3 bucket in Account B

```

Key Components:

1. **Trust Relationship:** Account B role trusts Account A
2. **Identity Policy:** Developer in Account A can assume role
3. **Role Policy:** Role has S3 permissions in Account B
4. **Temporary Credentials:** STS provides temporary tokens

Question 5: Least Privilege Implementation (Real-World)

Question

“A developer needs to deploy a Lambda function that reads from an S3 bucket, writes logs to CloudWatch, and calls an SNS topic. Design the least privilege IAM policy for this Lambda execution role.”

Expected Answer

Least Privilege Policy:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3ReadSpecificBucket",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::app-data-bucket",
        "arn:aws:s3:::app-data-bucket/input/*"
      ]
    }
  ]
}

```



```

    ],
  },
  {
    "Sid": "CloudWatchLogs",
    "Effect": "Allow",
    "Action": [
      "logs:CreateLogGroup",
      "logs:CreateLogStream",
      "logs:PutLogEvents"
    ],
    "Resource":
      "arn:aws:logs:us-east-1:123456789012:log-group:/aws/lambda/my-
function:*"
  },
  {
    "Sid": "SNSPublish",
    "Effect": "Allow",
    "Action": "sns:Publish",
    "Resource":
      "arn:aws:sns:us-east-1:123456789012:notifications-topic"
  }
]
}

```

Analysis of Least Privilege:

1. **S3:** Only GetObject and ListBucket (not DeleteObject)
2. **Resources:** Specific buckets and prefixes (not *:*)
3. **Logs:** Only required log actions (CreateLogStream, PutLogEvents)
4. **SNS:** Only Publish action on specific topic
5. **No wildcard:** Avoid arn:aws:* or Action: *

Anti-Patterns to Avoid:

```

❑ WRONG (Too permissive)
{
  "Effect": "Allow",
  "Action": "s3:*",
  "Resource": "*"
}

```

```

❑ WRONG (Too broad)
{
  "Effect": "Allow",
  "Action": "logs:*",
  "Resource": "*"
}

```

```
✓ RIGHT (Least privilege)
{
  "Effect": "Allow",
  "Action": ["logs:PutLogEvents"],
  "Resource":
    "arn:aws:logs:us-east-1:123456789012:log-group:/aws/lambda/my-
function:*"
}
```

Best Practices

Security Best Practices

1. **Enable MFA:** On root account and privileged users
2. **Use Groups:** Manage permissions at group level
3. **Rotate Credentials:** Access keys every 90 days
4. **Least Privilege:** Grant only necessary permissions
5. **Regular Audits:** Review IAM policies and access
6. **CloudTrail:** Enable logging for all API calls
7. **Avoid Root Account:** Use only for setup and billing
8. **Use Roles:** Instead of long-lived credentials for services

Operational Best Practices

- Use AWS SSO for centralized access management
- Implement permission boundaries for delegated administration
- Enable resource-based policies for cross-account access
- Use session tags for dynamic permissions
- Regularly review unused credentials and users
- Document your IAM strategy and governance model

Conclusion

AWS IAM is the foundation of AWS security. Mastering IAM components (users, groups, roles, policies) and advanced features (permission boundaries, MFA) is critical for:

- Securing AWS accounts

- Meeting compliance requirements
- Implementing least privilege principle
- Managing large-scale AWS deployments
- Cross-account and hybrid access management

Key Takeaway: Always start with a deny-all stance and explicitly grant only the permissions users need to perform their job.

References

[1] Amazon Web Services. (2025). AWS IAM User Guide. Retrieved from <https://docs.aws.amazon.com/iam/>

[2] Amazon Web Services. (2025). Permissions Boundaries for IAM Entities. Retrieved from https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_boundaries.html

[3] Amazon Web Services. (2025). IAM Security Best Practices. Retrieved from <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>

[4] Amazon Web Services. (2025). AWS Multi-Factor Authentication. Retrieved from https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_mfa.html